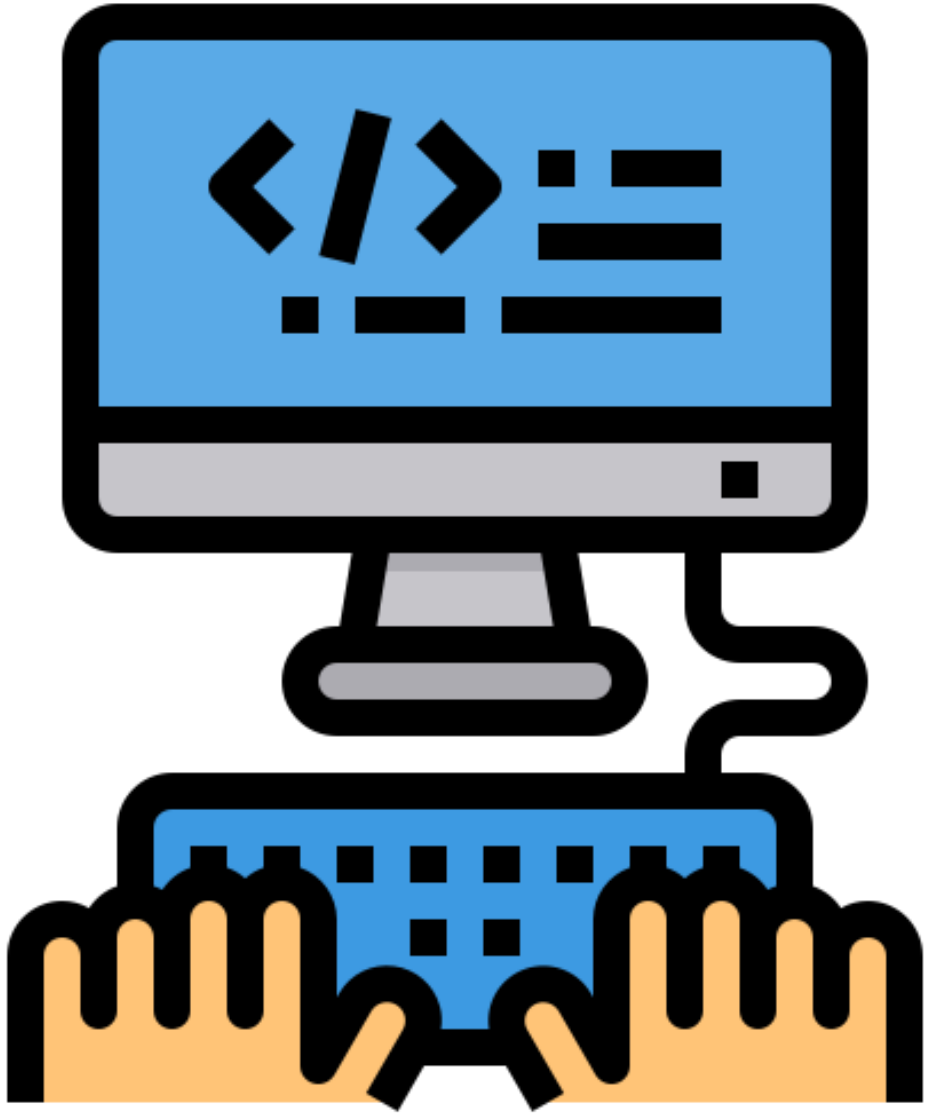
# CS 24 AP Computer Science A Review

## Week 3: Boolean Expression and if Statements

DR. ERIC CHOU
IEEE SENIOR MEMBER

# Topics

- Boolean Operators
- If-Statements
- If-Then-else statements
- Short-Circuit Evaluation
- Truth Table
- Boolean Algebra



# Boolean Operators

SECTION 1

| *a* | *!a* |
|---|---|
| true | false |
| false | true |

| *a* | *b* | *a && b* | *a \|\| b* |
|---|---|---|---|
| false | false | false | false |
| false | true | false | true |
| true | false | false | true |
| true | true | true | true |

# Boolean Operators

| Operator | Logical Meaning |
|---|---|
| && | Logical and |
| \|\| | Logical or |
| ! | Logical not |
| == | Is equal to |
| != | Is not equal to |

# Multiple Choice (1)

**Consider the following code:**

```
int x = 0;
if (x == 0)
    System.out.print("1");
else
    System.out.print("2");
    System.out.print("3");
```

Which the following best describes the result of executing the code segment?

# Multiple Choice (1)

Ⓐ Since the value of x is 0. the first print statement will be performed. producing 1 as the output.

Ⓑ Since the value of x is 0. the first print statement will be performed. producing 13 as the output.

Ⓒ Since the value of x is 0. the first print statement will be performed. producing 123 as the output.

Ⓓ == is not the correct Boolean operator, so a syntax error will be produced by the compiler prior to execution.

# Multiple Choice (1)

Ⓐ Since the value of x is 0. the first print statement will be performed. producing 1 as the output.

Ⓑ Since the value of x is 0. the first print statement will be performed. producing 13 as the output.

Ⓒ Since the value of x is 0. the first print statement will be performed. producing 123 as the output.

Ⓓ == is not the correct Boolean operator, so a syntax error will be produced by the compiler prior to execution.

# Multiple Choice (2)

Consider the following boolean expression

```
(X && !Y) || (!X && Y)
```

Which of the following condition(s) will always cause the expression to evaluate to true?

Ⓐ X is true

Ⓑ Y is false

Ⓒ X and Y have the same truth values

Ⓓ X and Y have the opposite truth values

# Multiple Choice (2)

Consider the following boolean expression

```
(X && !Y) || (!X && Y)
```

Which of the following condition(s) will always cause the expression to evaluate to true?

Ⓐ X is true

Ⓑ Y is false

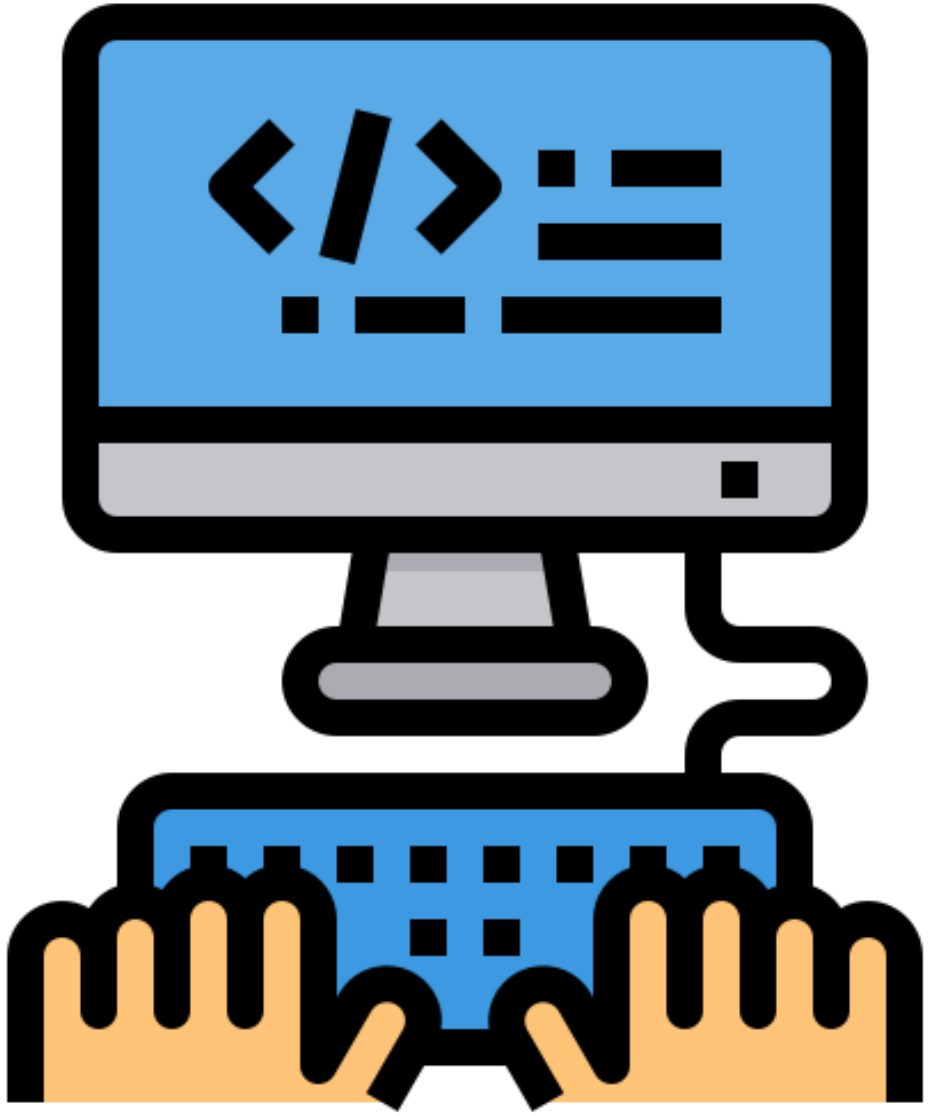
Ⓒ X and Y have the same truth values

Ⓓ X and Y have the opposite truth values

# Equivalent Logic Expressions

There are two ways to find an equivalent logic expressions:

1. Boolean Algebra (use logic simplification)
2. Truth Table (Listing all possible logic combination and its results)



# Boolean Algebra

SECTION 2

# Set Theory and Logic Design (only positive numbers are shown)

boolean m5 = ( x % 5 == 0);  // Set S5 = [0, 5, 10, 15, 20, …]

boolean m2 = ( x % 2 == 0);  // Set S2 = [0, 2, 4, 6, 8, 10, …]

| Boolean Expression | Set | Set Components |
|---|---|---|
| m5 && m2 | $S5 \cap S2$ | [0, 10, 20, 30, 40, …] |
| m5 \|\| m2 | $S5 \cup S2$ | [0, 2, 4, 5, 6, 8, 10, 12, 14, 15, …] |
| m2 && !m5 | $S2 - S5$ | [2, 4, 6, 8, 12, 14, 16, 18, …] |
| m5 && !m2 | $S5 - S2$ | [5, 15, 25, 35, 45, …] |
| m2 ^ m5 | $S5 \oplus S2$ | [2,4, 5, 6, 8, 12, 14, 15, 16, …] |

# All 16 Boolean Functions can be expressed by (x, y, !, &&, ||)

| | | | | |
|---|---|---|---|---|
| $x$ | 0 | 0 | 1 | 1 |
| $y$ | 0 | 1 | 0 | 1 |
| $0$ | 0 | 0 | 0 | 0 |
| $x \cdot y$ | 0 | 0 | 0 | 1 |
| $x \cdot y'$ | 0 | 0 | 1 | 0 |
| $x$ | 0 | 0 | 1 | 1 |
| $x' \cdot y$ | 0 | 1 | 0 | 0 |
| $y$ | 0 | 1 | 0 | 1 |
| $x \cdot y' + x' \cdot y$ | 0 | 1 | 1 | 0 |
| $x+y$ | 0 | 1 | 1 | 1 |
| $(x+y)'$ | 1 | 0 | 0 | 0 |
| $x \cdot y + x' \cdot y'$ | 1 | 0 | 0 | 1 |
| $y'$ | 1 | 0 | 1 | 0 |
| $x+y'$ | 1 | 0 | 1 | 1 |
| $x' \cdot y$ | 1 | 1 | 0 | 0 |
| $x'+y$ | 1 | 1 | 0 | 1 |
| $(x \cdot y)'$ | 1 | 1 | 1 | 0 |
| $1$ | 1 | 1 | 1 | 1 |

| Boolean Functions in Java | |
|---|---|
| boolean f0 = false; | boolean f8 = !(x+y); |
| boolean f1 = x && y; | boolean f9 = x && y + !x && !y; |
| boolean f2 = x && !y; | boolean f10 = !y; |
| boolean f3 = x; | boolean f11 = x + !y; |
| boolean f4 = !x && y; | boolean f12 = !x && y; |
| boolean f5 = y; | boolean f13 = !x + y; |
| boolean f6 = x && !y + !x && y; | boolean f14 = !(x && y) |
| boolean f7 = x + y; | boolean f15 = true; |

# Introduction

- A number of rules can be derived from these relations as Table 1 demonstrates.

- **Rule 1: X = 0 or X = 1**
- **Rule 2: 0 * 0 = 0**
- **Rule 3: 1 + 1 = 1**
- **Rule 4: 0 + 0 = 0**
- **Rule 5: 1 * 1 = 1**
- **Rule 6: 1 * 0 = 0 * 1 = 0**
- **Rule 7: 1 + 0 = 0 + 1 = 1**

**Table 1: Boolean Postulates**

# Laws of Boolean Algebra

- Table 2 shows the basic Boolean laws.
- Note that every law has two expressions, (a) and (b). This is known as *duality*.
- These are obtained by changing every AND(*) to OR(+), every OR(+) to AND(*) and all 1's to 0's and vice-versa.
- It has become conventional to drop the * (AND symbol) i.e. A.B is written as AB.

# Boolean Algebra Laws

**T1 : Commutative Law**

(a) $A + B = B + A$

(b) $A\,B = B\,A$

**T2 : Associate Law**

(a) $(A + B) + C = A + (B + C)$

(b) $(A\,B)\,C = A\,(B\,C)$

**T3 : Distributive Law**

(a) $A\,(B + C) = A\,B + A\,C$

(b) $A + (B\,C) = (A + B)\,(A + C)$

**T4 : Identity Law**

(a) $A + A = A$

(b) $A\,A = A$

**T5 :**

(a) $AB + A\overline{B} = A$

(b) $(A+B)(A+\overline{B}) = A$

**T6 : Redundance Law**

(a) $A + A\,B = A$

(b) $A\,(A + B) = A$

**T7 :**

(a) $0 + A = A$

(b) $0\,A = 0$

**T8 :**

(a) $1 + A = 1$

(b) $1\,A = A$

**T9 :**

(a) $\overline{A} + A = 1$

(b) $\overline{A}\,A = 0$

**T10 :**

(a) $A + \overline{A}\,B = A + B$

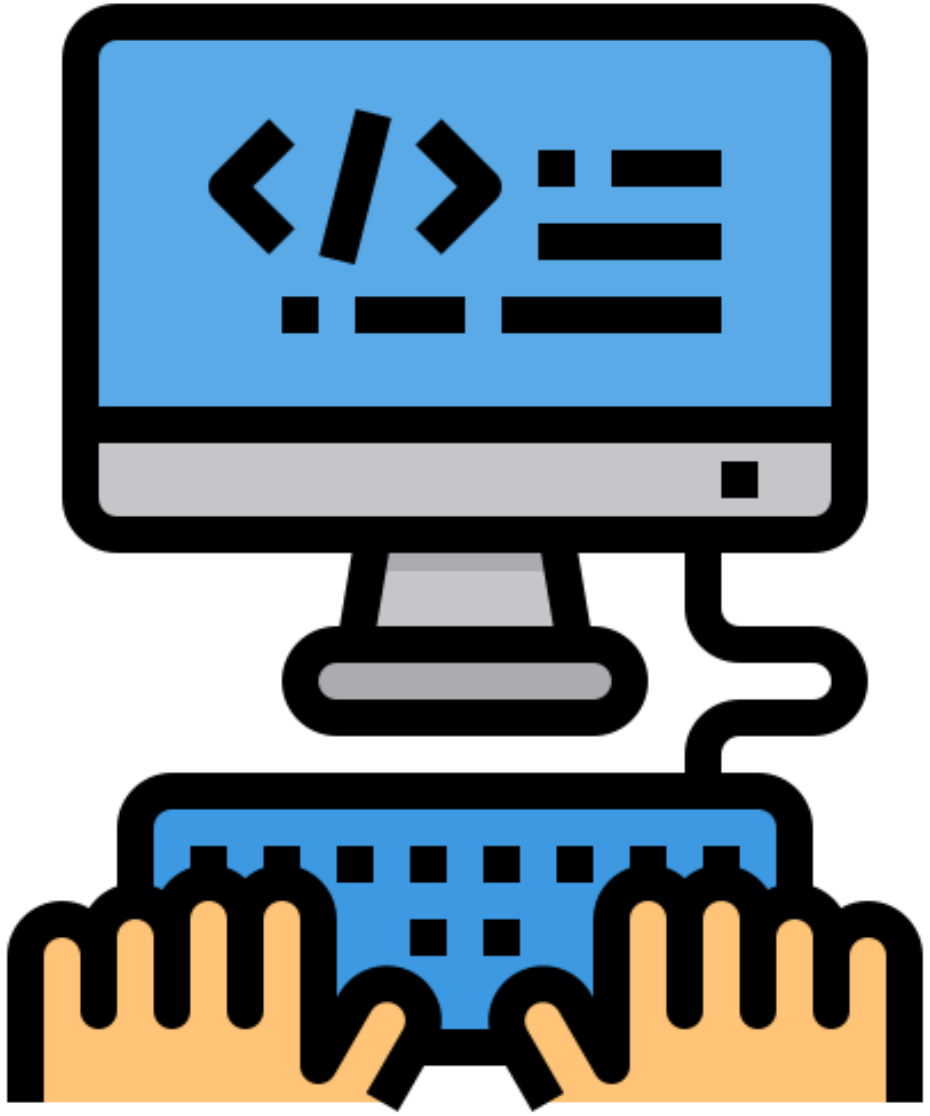
(b) $A\,(\overline{A} + B) = A\,B$

**T11 : De Morgan's Theorem**

(a) $\overline{(A + B)} = \overline{A}\;\overline{B}$

(b) $\overline{(A\,B)} = \overline{A} + \overline{B}$

| Name | AND form | OR form |
|---|---|---|
| Identity law | $1A = A$ | $0 + A = A$ |
| Null law | $0A = 0$ | $1 + A = 1$ |
| Idempotent law | $AA = A$ | $A + A = A$ |
| Inverse law | $A\bar{A} = 0$ | $A + \bar{A} = 1$ |
| Commutative law | $AB = BA$ | $A + B = B + A$ |
| Associative law | $(AB)C = A(BC)$ | $(A + B) + C = A + (B + C)$ |
| Distributive law | $A + BC = (A + B)(A + C)$ | $A(B + C) = AB + AC$ |
| Absorption law | $A(A + B) = A$ | $A + AB = A$ |
| De Morgan's law | $\overline{AB} = \bar{A} + \bar{B}$ | $\overline{A + B} = \bar{A}\bar{B}$ |



# Truth Table

SECTION 3

# Truth Table

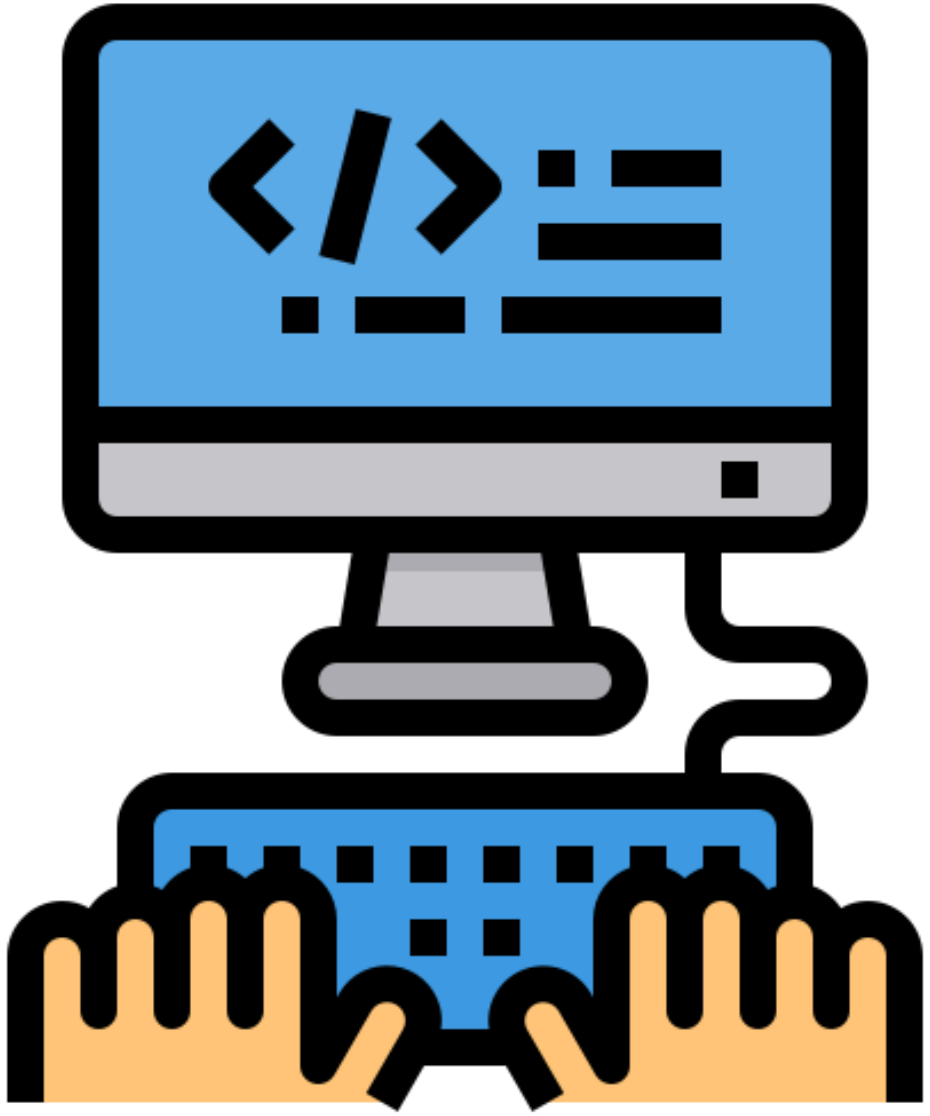
There are two ways to prove Boolean Algebra equations: Venn Diagram or Truth table.

Truth table is tedious, yet it the optimal way of proving the logic results.

How to evaluate (p v r) V –(p ^ q)?

Note: v is or, ^ is and

| $p$ | $q$ | $r$ | $p \wedge q$ | $\sim(p \wedge q)$ | $(p \vee r)$ | $(p \vee r) \vee \sim(p \wedge q)$ |
|---|---|---|---|---|---|---|
| T | T | T | T | F | T | T |
| T | T | F | T | F | T | T |
| T | F | T | F | T | T | T |
| T | F | F | F | T | T | T |
| F | T | T | F | T | T | T |
| F | T | F | F | T | F | T |
| F | F | T | F | T | T | T |
| F | F | F | F | T | F | T |



# If-Statements

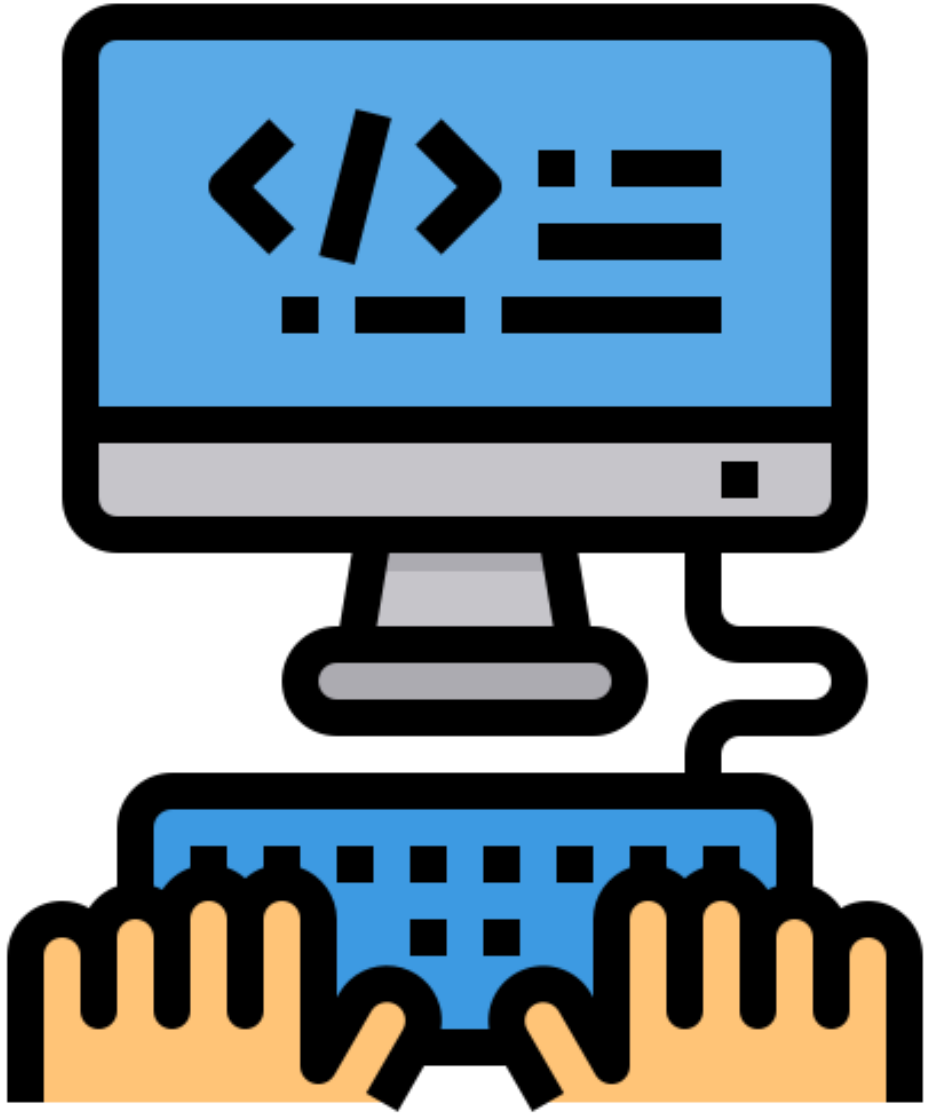
SECTION 4

## Condition is true

```
int number = 10;

if (number > 0) {
  // code
}

// code after if
```

## Condition is false

```
int number = 10;

if (number < 0) {
  // code
}

// code after if
```



# If-Then-Else Statements

SECTION 5

```java
if (score >= 90.0)
  System.out.print("A");
else
  if (score >= 80.0)
    System.out.print("B");
  else
    if (score >= 70.0)
      System.out.print("C");
    else
      if (score >= 60.0)
        System.out.print("D");
      else
        System.out.print("F");
```

(a)

Equivalent

This is better

```java
if (score >= 90.0)
  System.out.print("A");
else if (score >= 80.0)
  System.out.print("B");
else if (score >= 70.0)
  System.out.print("C");
else if (score >= 60.0)
  System.out.print("D");
else
  System.out.print("F");
```

(b)

# Conditional Statements

**The first 3-character substring, if the string shorter than 3, use the whole string:**

```
String s3 = str.length() >= 3 ?
            str.substring(0, 3) : str;
```

Use only AP-Subset:

```
String s3 = str;
if (s3.length()>=3) s3 = str.substring(0, 3);
```

# Dangling If

## else belongs to closest if

The else clause matches the most recent if clause in the same block.

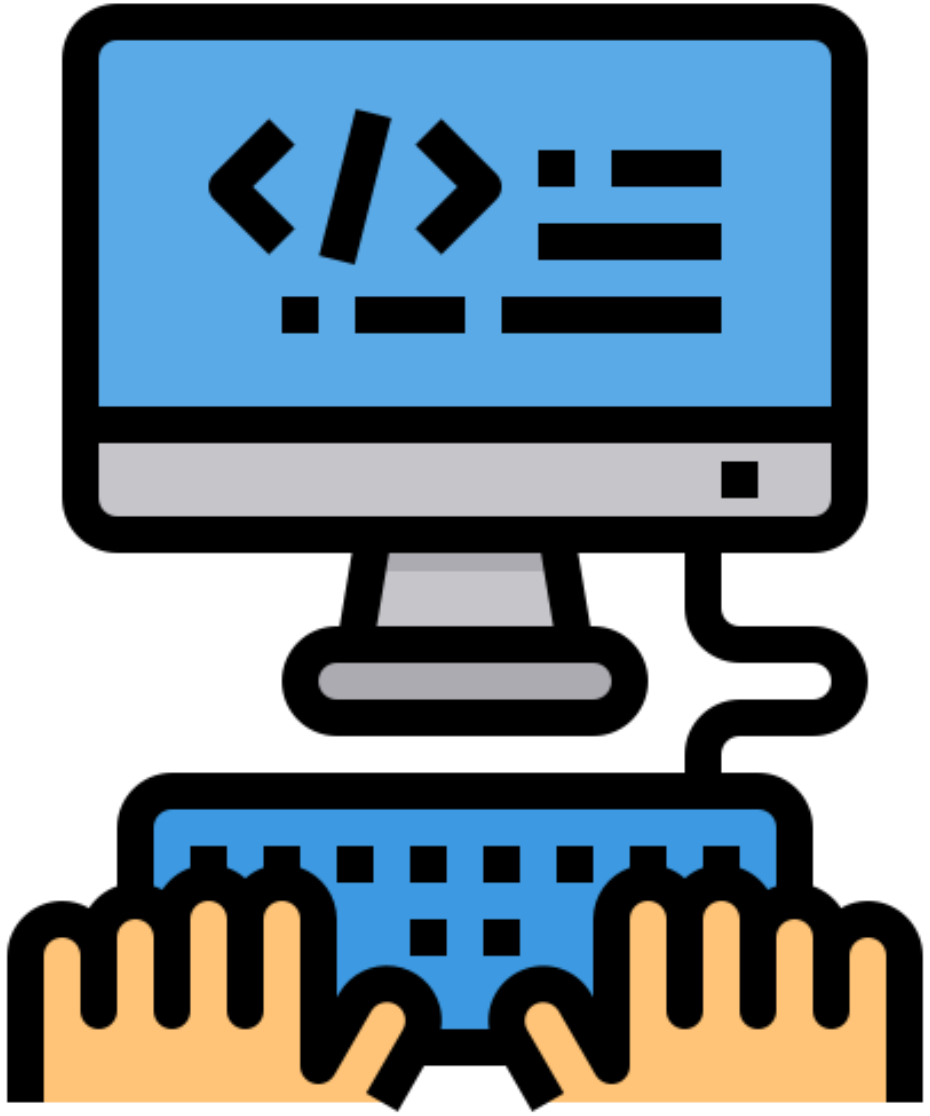
```
int i = 1;
int j = 2;
int k = 3;

if (i > j)
  if (i > k)
    System.out.println("A");
else
    System.out.println("B");
```

(a)

Equivalent

```
int i = 1;
int j = 2;
int k = 3;

if (i > j)
  if (i > k)
    System.out.println("A");
  else
    System.out.println("B");
```

(b)



# Short-Circuit Evaluation

SECTION 6

# Java Short-Circuit Evaluation

- The Java standard mandates left-to-right evaluation and a sequence point after the first operand for && and ||.
- Logical AND (&&): If the left operand evaluates to false, the entire expression is already known to be false. The right operand is not evaluated.
- Logical OR (||): If the left operand evaluates to true, the entire expression is already known to be true. The right operand is not evaluated.

# Short-Circuit AND Example:

```
int age = 20;
int hasID = 1; // 1 = true, 0 = false
if (age >= 18 && hasID) {
   System.out.println("Access granted: You can enter the club.");
}
```

```
int age = 2;
int hasID = 1; // 1 = true, 0 = false
if (age >= 18 && hasID) {
   System.out.println("Access granted: You can enter the club.");
}
```

# Short-Circuit OR Example:

```
int age = 2;
if (age <18 || age>65) {
   System.out.println("You got discount. ");
}
```

```
int age = 20;
if (age <18 || age>65) {
 System.out.println("You got discount. ");
}
```

# Short-Circuit Evaluation

## Week1/ShortCircuit.java

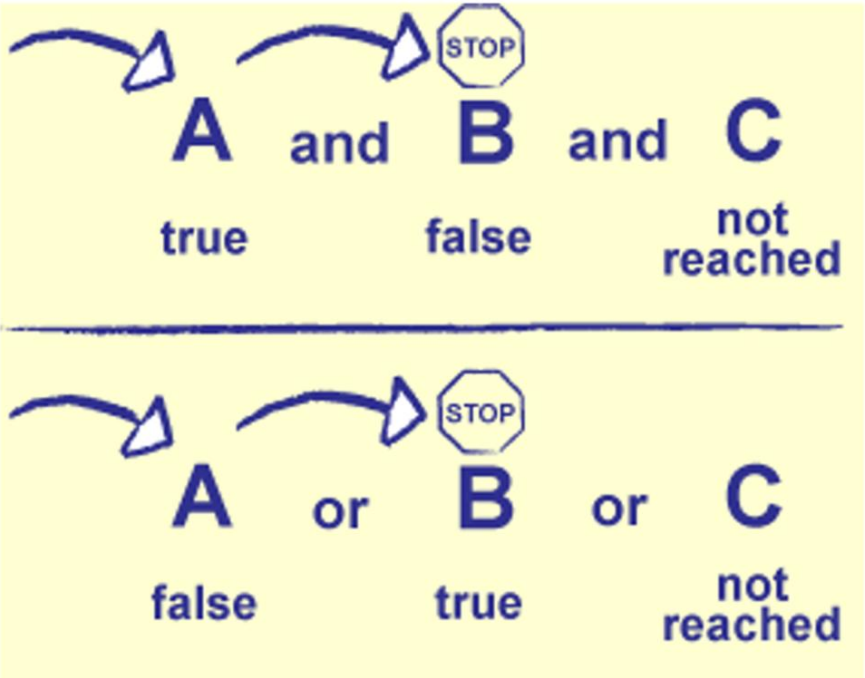
```java
public class ShortCircuit{
  public static void main(String[] argv) {
    int denom = 0;
    int num = 3;
    if (denom != 0 && num / denom > 10) {
      System.out.println("Here");
    } else {
      System.out.println("There");
    }
  }
}
```

**Short-Circuit Example:**

int count =0;

int total = 0;

boolean result = (count !=0 && **total/count >0**);



# Multiple Choice (3)

**Consider the following code:**

```
boolean a = true , b = false ;
if (/* missing code */)
    System.out.print("NIce job.");
else
    System.out.print("Nicer job.");
```

which of the following could be used to replace /* missing code */ so that the output of this block of code is “Nicer job.”?

```
Ⅰ. a && !b
Ⅱ. !a || b
Ⅲ. !a && b
```

# Multiple Choice (3)

Ⓐ Ⅰ only

Ⓑ Ⅰ and Ⅱ only.

Ⓒ Ⅰ and Ⅲ only.

Ⓓ Ⅱ and Ⅲ only.

# Multiple Choice (3)

Ⓐ Ⅰ only

Ⓑ Ⅰ and Ⅱ only.

Ⓒ Ⅰ and Ⅲ only.

Ⓓ Ⅱ and Ⅲ only.

# Multiple Choice (4)

Suppose p and q are declared as Boolean variables and have been initialized to unknown truth values.

Which of the following Boolean expression evaluate to?

```
(!p && !q) || !(p || q)
```

# Multiple Choice (4)

Ⓐ The expression always evaluates to true.

Ⓑ The expression always evaluates to false.

Ⓒ The expression evaluates to true whenever p is false.

Ⓓ The expression evaluates to false whenever p and q have opposite truth values.

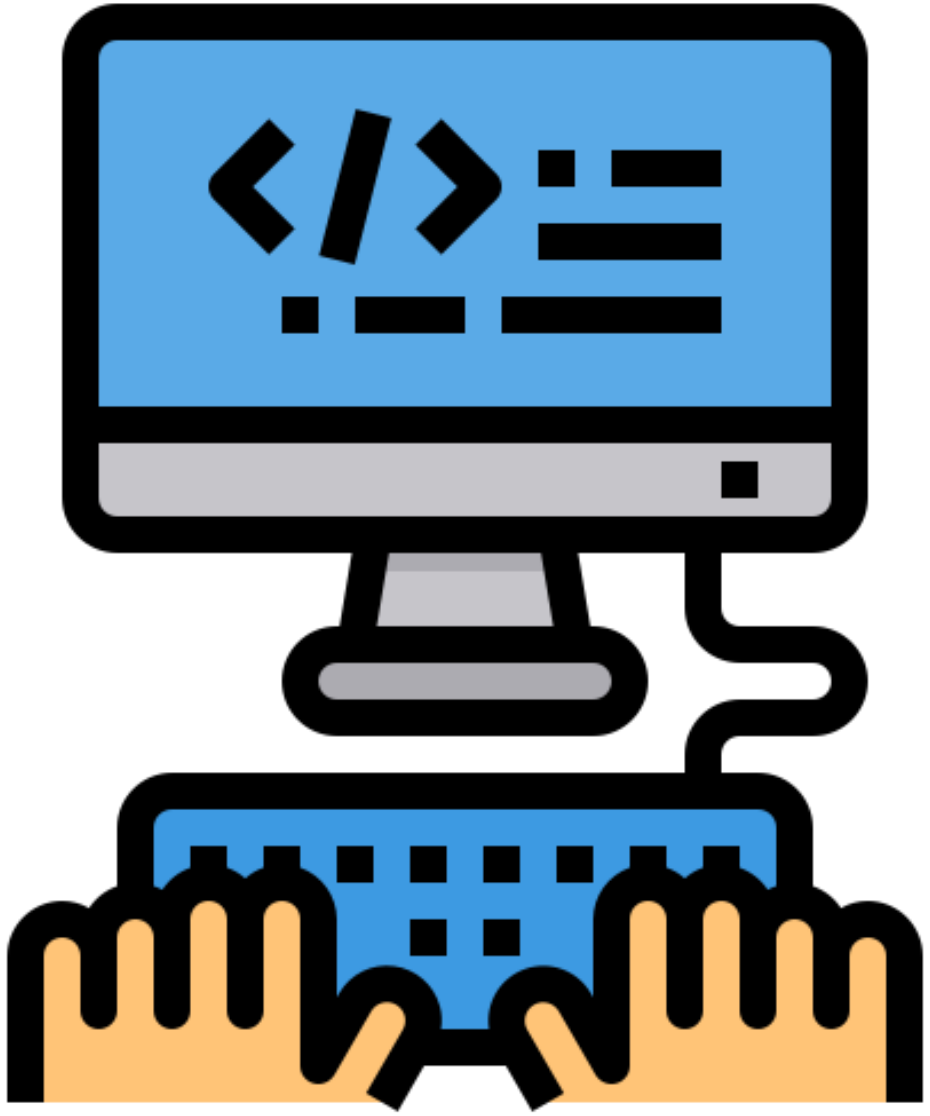
# Multiple Choice (4)

Ⓐ The expression always evaluates to true.

Ⓑ The expression always evaluates to false.

Ⓒ The expression evaluates to true whenever p is false.

Ⓓ The expression evaluates to false whenever p and q have opposite truth values.



# Practice MCQ Questions

SECTION 7

# Multiple Choice (1)

Consider the following code segment:

```
for (int i=200; i>0; i/=3){
    if (i%2==0)
      System.out.print(i+" ");
}
```

Which of the output as result fo executing the code segment?

# Multiple Choice (1)

Ⓐ 200 66 22 7 2

Ⓑ 66 22 7 2

Ⓒ 200 66 22 2

Ⓓ 7

# Multiple Choice (1)

Ⓐ 200 66 22 7 2

Ⓑ 66 22 7 2

Ⓒ 200 66 22 2

Ⓓ 7

# Multiple Choice (2)

Suppose p and q are declared as boolean variables and have been initialized to unknown truth values.

Which does the following Boolean expression evaluate to?

```
(p && !q) || (p && q)
```

# Multiple Choice (2)

Ⓐ The expression always evaluates to the same value as p.

Ⓑ The expression always evaluates to the same value as q.

Ⓒ The expression always evaluates to false.

Ⓓ The expression evaluates to true only if p and q have opposite truth values.

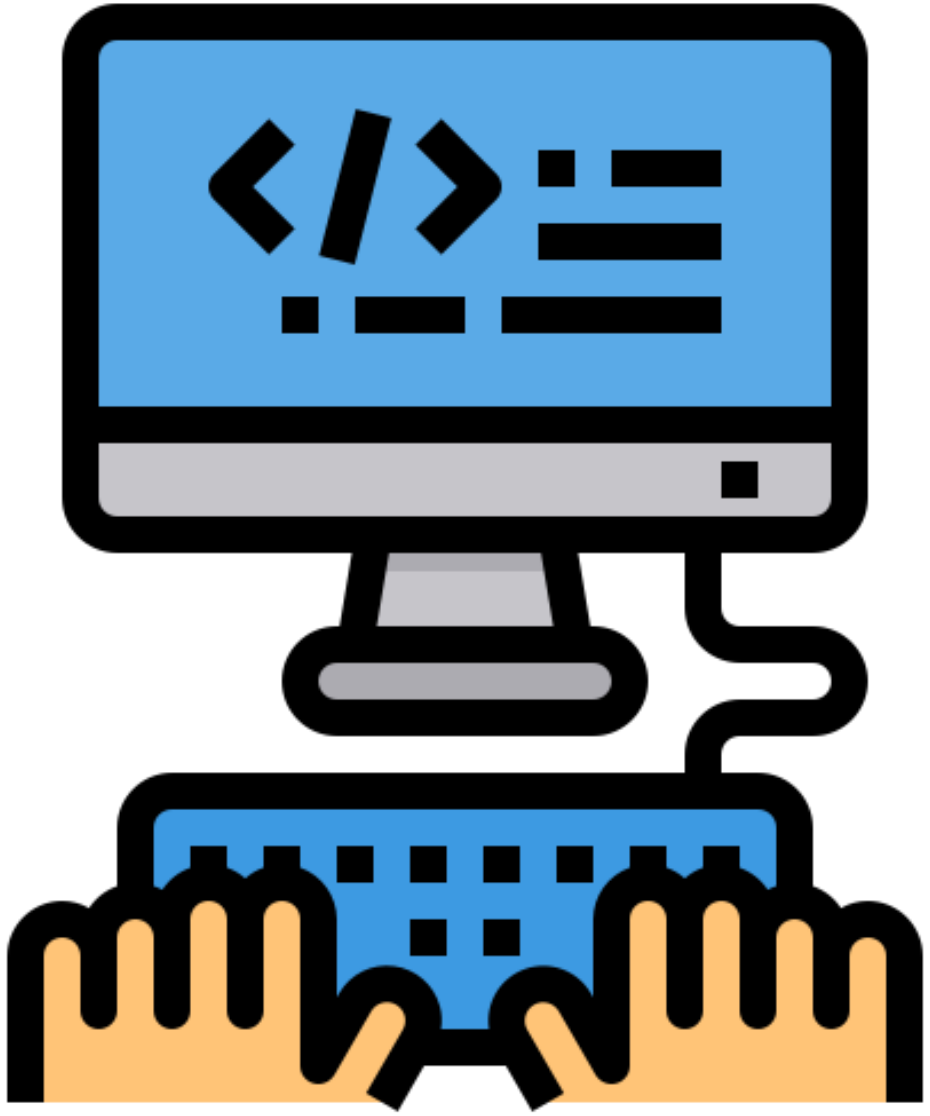
# Multiple Choice (2)

Ⓐ The expression always evaluates to the same value as p.

Ⓑ The expression always evaluates to the same value as q.

Ⓒ The expression always evaluates to false.

Ⓓ The expression evaluates to true only if p and q have opposite truth values.



# Logic Algorithms

SECTION 8

# Probability

Create an integer which has 30% chance to be 0 and 70% chance to be 1.

```
int a = 1;
if (Math.random() < 0.3) a = 0;
```

# Leap Year

```
public boolean isLeapYear(int y){
    if (y % 400 == 0) return true;
    if (y %100 == 0) return false;
    return y % 4 == 0;
}
```

```
public boolean isLeapYear(int y){
    return y % 400 == 0 ||
           y %100 != 0 && y % 4 == 0;
}
```

# For All,

Check if an array **`ary`** has all even numbers:

```
public boolean allEven(int[] ary){
   for (int i=0; i<ary.length; i++){
      if (ary[i] % 2 == 1) return false;
   }
    return true;
}
```

# There exists,

Check if an array **`ary`** has some even numbers:

```
public boolean hasEven(int[] ary){
   for (int i=0; i<ary.length; i++){
      if (ary[i] % 2 == 0) return true;
   }
   return false;
}
```